

This will give you the IP address of interfaces inside the container, rather than your host.

LXC also supports cloning and snapshots of the container. To clone a container, use the following command:

```
$sudo lxc-copy -n mycontainer -N newcontainer
```



Note: To clone your container, it should not be in a running state.

We will discuss snapshots in the next section.

Configuring Linux containers

Till now, we have used LXC tools only for general purposes. But when you are using containers in a production or development environment, you need more controls and options to get your work done. To configure a container, you must know how to use the *config* file. But, first, you need to know the actual location of your containers on your disk. The answer is at the path */var/lib/lxc/your_container_name*. Inside this path, you will find the *rootfs* directory, which is the root of your container (also called the backing store), and a *config* file. You can change this container's path while creating the container using *lxc-create* by passing parameter *-P newpath* or *-lxcpath=PATH*.

You can even create a separate partition and use it as the root file system of the container. The default backing store, which is */var/lib/lxc/container_name/rootfs* is called *dir backing store*. Other available options are *lvm*, *loop*, *brtfs*, *zfs*, *aufs*, *overlays* and *rbd*. With the *-P* option you can only define the path of your container's *rootfs*, but the container will still be a *dir* container.

So, to change the backing store, use *-B backing_store_type* with *lxc-create* and *lxc-clone* (to clone it into another container of a different backing store).

A strong feature that LXC comes with is the container's snapshot capability. You can create a snapshot using *lxc-snapshot*.

```
$sudo lxc-snapshot -n mycontainer
```



Note: Snapshots are not supported for *dir* based containers; so before using them, you have to clone them to the container of type *aufs* or *overlays*.

This will save the snapshot with the name *snap0*. Now, whenever you do some changes you are not sure about in your container, you can just create a snapshot before the change and restore it later with the following command:

```
$sudo lxc-snapshot -r snap0 -n mycontainer.
```

To list all the snapshots of a container, use the following command:

```
$sudo lxc-snapshot -L -n mycontainer
```

```
(Mon Jul-7 4:24:17pm)-(CPU 10.28%Net 24%)-(hackintosh)-(4.2G:82)
> sudo lxc-copy -n mycontainer -B newcontainer -B overlayfs -s
(Mon Jul-7 5:05:33pm)-(CPU 10.28%Net 22%)-(hackintosh)-(4.2G:82)
> lxc-ls
(Mon Jul-7 5:05:40pm)-(CPU 10.28%Net 23%)-(hackintosh)-(4.2G:82)
> sudo lxc-ls
basebox1 busybox1 devstack mycontainer newcontainer ubuntu2
(Mon Jul-7 5:05:45pm)-(CPU 10.28%Net 22%)-(hackintosh)-(4.2G:82)
> sudo lxc-snapshot -n newcontainer
(Mon Jul-7 5:07:02pm)-(CPU 10.18%Net 22%)-(hackintosh)-(4.2G:82)
> sudo lxc-snapshot -L -n newcontainer
snap1 (/var/lib/lxc/newcontainer/snap0) 2017:07:03 17:06:56
snap1 (/var/lib/lxc/newcontainer/snap0) 2017:07:03 17:07:02
(Mon Jul-7 5:07:20pm)-(CPU 10.18%Net 23%)-(hackintosh)-(4.2G:82)
> sudo lxc-snapshot -r snap1 -n newcontainer
```

Figure 5: Basic snapshot operations

Every container has a *config* file at path */var/lib/lxc/your_container/config*. You can use this file for almost every possible change you want to do with LXC, including but not limited to networking, setting cgroups limitations, profiling and other common container related configurations. For example, you can mount an external folder inside the container by adding the following line in the *config* file of your container:

```
lxc.mount.entry = /mnt/share /root/share none ro,bind 0.0
```

We will discuss more about the *config* file in the next article in this series.

How LXC is different from Docker and VMs

LXC (or LXD) has been created to replace the work of VMs as the latter are really heavy to use and can be unmanageable if run in large numbers. The only difference between containers and VMs is that the former have the same kernel layer as that of the host, whereas VMs provide full isolation.

So Linux containers run a whole Linux machine (or simply multiple services) inside the isolated environment provided by the Linux kernel, whereas Docker's are replacements of the traditional way of running applications and run them in isolated environments, i.e., Docker containers are made to run a single application inside their containers. LXC containers are even capable of nested containerisation, which means you can run Docker or any other container inside an LXC container without any issues. In general, it is immaterial what is running inside a container, since all applications will be treated the same way. We will discuss that comparison in the next article in this series.

By: Shubham Dubey

The author is a Linux enthusiast, and works in the fields of cloud computing, virtualisation and cyber security. He is also an active contributor to some large cloud and virtualisation projects like Open Stack and Ovirt. He can be contacted at sdubey504@gmail.com.